Quick Sort

1. Source Code (C)

```c
#include<stdio.h>

// Function to swap two elements in the array
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Function to partition the array and return the index of the pivot
int partition(int arr[], int low, int high) {
    int pivot = arr[high];

    // Choosing the last element as the pivot
    int i = low - 1;

    // Index of smaller element
    for (int j = low; j < high; j++) {
        // If current element is smaller than the pivot
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    // Place the pivot element in its correct position
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

// Function to implement Quick Sort
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        // Partitioning index
        int pi = partition(arr, low, high);

        // Recursive sorting of the sub-arrays
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// Function to print the array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {12, 7, 11, 13, 5, 6};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array:\n");
    printArray(arr, n);

    quickSort(arr, 0, n - 1);

    printf("Sorted array:\n");
    printArray(arr, n);
```

```
    return 0;
}
```

2. Program Output

```
Original array:
12 7 11 13 5 6

Sorted array:
5 6 7 11 12 13
```